

1. **Tables and keys:** Why a database? · Tables, rows, columns and types · Primary keys · Foreign keys · One-to-many and many-to-many · Constraints: NOT NULL, UNIQUE, CHECK, DEFAULT
2. **Reading and changing rows:** SELECT: asking for rows · INSERT, UPDATE, DELETE: CRUD in SQL · UPDATE and DELETE need a WHERE · NULL: the missing value · Pagination: LIMIT, OFFSET and ORDER BY
3. **Joins and aggregates:** JOIN: combining tables · LEFT JOIN: keeping rows without a match · COUNT and GROUP BY
4. **Indexes and migrations:** Indexes: finding rows fast · Migrations: versioning the schema
5. **From Node to SQL:** Connecting from Node: drivers and pools · SQL injection and parameterised queries · Transactions: all or nothing · The models layer: where the SQL lives · From rows to JSON

Tables and keys

Why a database?

A database is a separate program that stores the application's data on disk, lets many requests read and change it safely at the same time, and answers questions about it in a query language.

- Persistence** The data is on disk, not in the server's memory, so a restart loses nothing.
- Concurrency** Two requests changing the same data at once, or two copies of the server side by side, see one consistent version: the database orders and isolates the changes.
- Querying** You describe *what* you want ("the open tasks of user 3, newest first, 10 per page") and the database works out *how* to find it, even among millions of rows.
- Integrity** The database itself refuses data that breaks the rules you declared, such as "every task belongs to a user that exists".

Example. With the array: you POST /tasks three times, edit app.js, node --watch restarts, and GET /tasks returns []. With PostgreSQL behind the same routes, the restart changes nothing: GET /tasks runs SELECT ... FROM tasks and the three rows are still there. Run two copies of the server (on ports 3000 and 3001) and both see the same tasks, which two separate arrays never could.

Common mistake. Thinking a database is "just a bigger array" that you load whole into memory and filter in JavaScript ((await getAllTasks()).filter(...)). That throws away the point: let the database filter, sort, count and page (WHERE, ORDER BY, COUNT, LIMIT) and send back only the rows the request needs.

Tables, rows, columns and types

A table stores one kind of thing; each row is one item and each column is one property with a fixed data type. The set of tables, columns, types and rules is the schema.

Common column types

Kind of data	PostgreSQL	MySQL	Use it for
Whole number	INTEGER, BIGINT	INT, BIGINT	Counts, ids, foreign keys
Auto-numbered id	SERIAL or INTEGER GENERATED ALWAYS AS IDENTITY	INT AUTO_INCREMENT	The primary key
Text	TEXT, VARCHAR(n)	VARCHAR(n), TEXT	Titles, emails, names
True / false	BOOLEAN	BOOLEAN (stored as TINYINT(1))	Flags such as done
Exact decimal	NUMERIC(10,2)	DECIMAL(10,2)	Money: never a float
Date / date and time	DATE, TIMESTAMP	DATE, DATETIME	Due dates, created_at

SQL (PostgreSQL)

```
CREATE TABLE tasks (
  id      INTEGER,
  user_id INTEGER,
  title   TEXT,
  done    BOOLEAN,
  due_date DATE      -- may be empty (NULL): not every task has a date
);
```

Example. The row (7, 3, 'Draft project plan', FALSE, '2026-10-12') fits this table: an id, the id of its owner, a title, a boolean and a date. INSERT INTO tasks (id, title, done) VALUES (8, 'Study', 'maybe') is refused, because 'maybe' is not a boolean. The columns an INSERT leaves out stay empty (NULL) until the table declares a rule for them.

Common mistake. Storing everything as text "to keep it simple": dates as '06/10/2026', prices as '12,50', flags as 'yes'. The database can then no longer sort dates correctly ('10/01' sorts before '9/30'), add prices or check that a flag is really a flag. Pick the type that matches the meaning.

Primary keys

A primary key is the column (or set of columns) whose value identifies exactly one row: it can never be empty (NULL) and never repeat.

Surrogate key A number with no meaning outside the database, generated as rows are inserted: SERIAL or the standard GENERATED ALWAYS AS IDENTITY in PostgreSQL, INT AUTO_INCREMENT in MySQL. Most tables use one.

UUID A random 128-bit value such as 3f2b...: ids cannot be guessed and can be created outside the database.

Composite key Two columns together: PRIMARY KEY (task_id, tag_id) in a join table allows each pair only once.

SQL (PostgreSQL)

```
CREATE TABLE users (
  id      SERIAL PRIMARY KEY,      -- 1, 2, 3... chosen by the database
  email   TEXT
);
-- MySQL: id INT AUTO_INCREMENT PRIMARY KEY
```

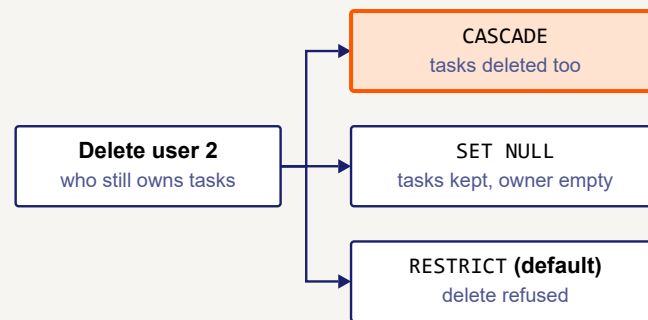
Example. The API receives POST /tasks with { "title": "Study" } and **does not** send an id: the database picks the next one (say 10) and reports it back (RETURNING id, see [INSERT, UPDATE, DELETE](#)), so the server can answer 201 Created with Location: /tasks/10. If ana later changes her email, users.id stays 1 and every task that points to her still works.

Common mistake. Using a value that can change or repeat as the key, such as the email or the task title, or computing the next id in JavaScript (tasks.length + 1, as an in-memory array version often does). Two requests arriving together would get the same number. Let the database generate ids.

Foreign keys

A foreign key is a column that stores the primary key of a row in another table (tasks.user_id → users.id); the

database then refuses any value that points to a row that does not exist.



ON DELETE decides what happens to the tasks of a deleted user.

SQL (PostgreSQL)

```

CREATE TABLE tasks (
  id      SERIAL PRIMARY KEY,
  user_id INT REFERENCES users(id) ON DELETE CASCADE,
  title   TEXT
);

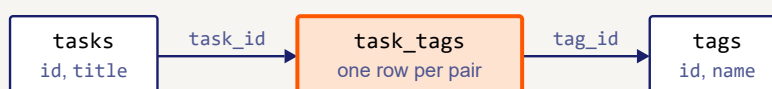
INSERT INTO tasks (user_id, title) VALUES (99, 'Ghost task');
-- ERROR: insert or update on table "tasks" violates foreign key constraint
DELETE FROM users WHERE id = 2;  -- also deletes every task with user_id = 2
  
```

Example. In the sample database ben (user 2) owns three tasks. `DELETE FROM users WHERE id = 2` removes ben **and** tasks 5, 6 and 9, because of `ON DELETE CASCADE`; their rows in `task_tags` go too, since that table cascades from tasks. Try the "ON DELETE CASCADE" example in the SQL runner.

Common mistake. Thinking REFERENCES is only documentation, or that the database "joins the tables for you". A foreign key is a **rule** checked on every change; reading the related rows together is a separate step that you write with JOIN.

One-to-many and many-to-many

In a **one-to-many** relationship the foreign key goes in the table on the "many" side; a **many-to-many** relationship needs a third **join table** that holds pairs of foreign keys.



A many-to-many relationship goes through a join table.

Where the reference goes

Relationship	Example	The reference
One-to-one	A user and their profile	A foreign key that cannot repeat (UNIQUE)
One-to-many	A user and their tasks	tasks.user_id, on the "many" side
Many-to-many	Tasks and tags; students and courses	A join table: task_tags (task_id, tag_id)

SQL (PostgreSQL)

```

CREATE TABLE tags (
  id      SERIAL PRIMARY KEY,
  name    TEXT NOT NULL UNIQUE
);

CREATE TABLE task_tags (
  task_id INT NOT NULL REFERENCES tasks(id) ON DELETE CASCADE,
  tag_id  INT NOT NULL REFERENCES tags(id) ON DELETE CASCADE,
  PRIMARY KEY (task_id, tag_id)  -- each pair at most once
);
  
```

```
-- task 7 has the tags 1 (urgent) and 3 (planning):
INSERT INTO task_tags (task_id, tag_id) VALUES (7, 1), (7, 3);
```

Example. In the sample database, the rows (7, 1) and (7, 3) of `task_tags` say "task 7 is tagged urgent and planning", and (1, 1), (7, 1) and (9, 1) say "urgent is on tasks 1, 7 and 9". To list a task's tags you join three tables (`tasks` → `task_tags` → `tags`): the SQL runner challenge Tasks tagged "urgent".

Common mistake. Storing a list inside one column, such as tags TEXT with 'urgent,study'. You can no longer check that each tag exists, rename a tag in one place, or find "all urgent tasks" without slow and fragile text matching (LIKE '%urgent%' also matches not-urgent). One fact per row: that is what the join table does.

Constraints: NOT NULL, UNIQUE, CHECK, DEFAULT

Constraints are rules written in the schema that the database enforces on every change: NOT NULL (a value is required), UNIQUE (no duplicates), CHECK (a condition must hold) and DEFAULT (the value used when none is given).

NOT NULL The column must have a value: title, email.

UNIQUE No two rows may share the value: a second account with the same email is refused. In MySQL a unique text column must be VARCHAR(255), because MySQL cannot index an unbounded TEXT.

CHECK (condition) Any rule about the row: CHECK (title <> ''), CHECK (price >= 0).

DEFAULT value What is stored when the INSERT leaves the column out: done BOOLEAN NOT NULL DEFAULT FALSE, created_at TIMESTAMP NOT NULL DEFAULT NOW().

When a constraint says no (PostgreSQL error codes)

Code	Meaning	Usual answer
23505	Unique violation	409 Conflict: "email already registered"
23503	Foreign-key violation	400 or 422
23502	Missing required value	400 or 422

SQL (PostgreSQL)

```
CREATE TABLE users (
  id          SERIAL PRIMARY KEY,
  email       TEXT NOT NULL UNIQUE,
  created_at  TIMESTAMP NOT NULL DEFAULT NOW()
);
CREATE TABLE tasks (
  id          SERIAL PRIMARY KEY,
  user_id     INT NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  title       TEXT NOT NULL CHECK (title <> ''),
  done        BOOLEAN NOT NULL DEFAULT FALSE
);
```

Example. INSERT INTO users (email) VALUES ('ana@example.com') gives ana an id and created_at = NOW() without naming them. Running it again fails with duplicate key value violates unique constraint "users_email_key" (code 23505), which the controller answers with 409 Conflict. Run the "Constraints say no" example in the SQL runner to meet each error.

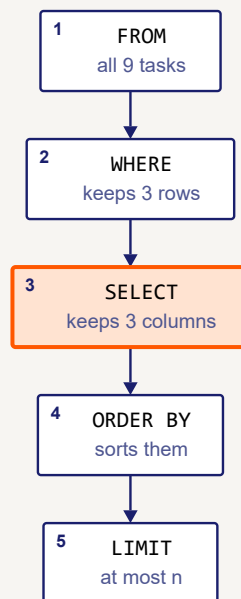
Common mistake. Relying on NOT NULL to reject empty text. An empty string '' is a value, not NULL, so title TEXT NOT NULL accepts ''. If empty titles are not allowed, add CHECK (title <> ''), and validate in the controller for a clear message.

Reading and changing rows

SELECT: asking for rows

SELECT reads data: you name the columns you want, the table (FROM), a condition the rows must meet (WHERE) and

the order (ORDER BY); the answer is always a new table of rows.



Written first, run third: the order in which the database works through a SELECT.

SELECT columns	The columns of the answer.
FROM table	Where the rows come from.
WHERE condition	Which rows to keep: =, <> (not equal), <, >=, AND, OR, NOT, IN (...), BETWEEN ... AND ..., LIKE '%text%' (% matches any characters).
ORDER BY column [ASC DESC]	The order of the rows.
LIMIT n	At most n rows.

SQL

```

SELECT id, title, due_date
FROM tasks
WHERE done = FALSE AND user_id = 1
ORDER BY id;

SELECT id, title FROM tasks WHERE title LIKE '%SQL%';
SELECT email FROM users WHERE id IN (1, 3);

```

Example. On the sample data, the first query returns tasks 1, 3 and 4 of ana (task 2 is done). Read it in the order it runs: FROM tasks (9 rows) → WHERE done = FALSE AND user_id = 1 (3 rows) → SELECT id, title, due_date (3 columns) → ORDER BY id. Load it in the SQL runner and change the WHERE to see the result change.

Common mistake. Writing SELECT * in application code. It returns every column, including ones you add later (a password_hash, say), and the API may send them to the client. In a model, name the columns you need: SELECT id, title, done FROM tasks.

INSERT, UPDATE, DELETE: CRUD in SQL

The four CRUD operations of an API map onto four SQL statements: **Create** → INSERT, **Read** → SELECT, **Update** → UPDATE, **Delete** → DELETE.

INSERT INTO table (columns) VALUES (values)	Adds a row; columns you leave out take their DEFAULT (or NULL).
UPDATE table SET column = value, ... WHERE condition	Changes the rows that match.
DELETE FROM table WHERE condition	Removes the rows that match.

RETURNING columns

PostgreSQL and SQLite: hands back the rows as stored, such as the new id after an INSERT. MySQL has none; mysql2 gives insertId and affectedRows instead.

From route to SQL to status code (Tasks API)

Route	SQL	Response
POST /tasks	INSERT INTO tasks (user_id, title) VALUES (\$1, \$2) RETURNING id, user_id, title, done	201 + the new task
GET /tasks	SELECT ... FROM tasks ORDER BY id	200 + an array (maybe empty)
GET /tasks/:id	SELECT ... FROM tasks WHERE id = \$1	200, or 404 if no row
PATCH /tasks/:id	UPDATE tasks SET ... WHERE id = \$1 RETURNING id, user_id, title, done	200, or 404 if 0 rows changed
DELETE /tasks/:id	DELETE FROM tasks WHERE id = \$1	204, or 404 if 0 rows changed

SQL (PostgreSQL)

```

INSERT INTO tasks (user_id, title)
VALUES (3, 'Write tests')
RETURNING id, user_id, title, done;      -- → 10 | 3 | Write tests | false

UPDATE tasks SET done = TRUE WHERE id = 10
RETURNING id, done;                      -- → 10 | true   (1 row changed)

DELETE FROM tasks WHERE id = 10;         -- 1 row changed
DELETE FROM tasks WHERE id = 10;         -- 0 rows changed → the API answers 404

```

Example. PATCH /tasks/5 with { "done": true } runs UPDATE tasks SET done = TRUE WHERE id = 5 RETURNING id, title, done: one row changes and comes back, so the controller sends 200 with it. PATCH /tasks/999 runs the same statement, zero rows change, rows is empty, and the controller sends 404.

Common mistake. Running SELECT after INSERT to "find the row I just added", for example by taking the highest id. Another request may have inserted a row in between, so you would return someone else's task. RETURNING (or insertId in MySQL) gives you exactly your row from the same statement.

UPDATE and DELETE need a WHERE

UPDATE and DELETE act on **every row that matches the WHERE**; with no WHERE at all they change or delete the whole table, and there is no undo.

1. Write the WHERE first, then the rest of the statement.
2. Preview the rows with a SELECT that uses the same WHERE.
3. Run it and check the count the driver reports (rowCount): UPDATE ... WHERE id = \$1 must change 0 or 1 row. For a risky change by hand, use a transaction and ROLLBACK if the count is wrong.

SQL

```

-- 1. preview: which rows would change?
SELECT id, title FROM tasks WHERE user_id = 2 AND done = TRUE;
-- 2. change exactly those
DELETE FROM tasks WHERE user_id = 2 AND done = TRUE;  -- 1 row changed

-- The accident:
UPDATE tasks SET done = TRUE;      -- 9 rows changed: every task is now done

```

Example. The preview shows one row (task 6, "Review pull request"), so the DELETE with the same WHERE reports "1 row changed": the count you expected. Load "UPDATE without WHERE" in the SQL runner to see the accident: the runner warns you, and because every run starts from a fresh copy, nothing is really lost there.

Common mistake. Believing a narrow-looking condition is narrow. `DELETE FROM tasks WHERE title LIKE '%test%'` also deletes "Contest entry" and "Latest notes". Preview with `SELECT` and read the count before changing anything that is not selected by its key.

NULL: the missing value

NULL means "no value / unknown". It is not 0, not '' and not false; any comparison with NULL is itself unknown, so you test for it with `IS NULL` and `IS NOT NULL`.

IS NULL, IS NOT NULL The only reliable tests for a missing value.

COALESCE(a, b, ...) The first argument that is not NULL: `COALESCE(due_date, 'no date')`.

COUNT(column) Skips NULLs: `COUNT(due_date)` counts the tasks that have a date, `COUNT(*)` counts rows.

In JavaScript pg turns SQL NULL into `null`.

SQL (PostgreSQL)

```
SELECT id FROM tasks WHERE due_date = NULL;      -- always 0 rows
SELECT id FROM tasks WHERE due_date IS NULL;     -- the tasks with no date
SELECT id, COALESCE(due_date, 'no date') AS due FROM tasks;

-- PATCH: change only what was sent ($3 is NULL when "done" was not sent)
UPDATE tasks
SET title = COALESCE($2, title),
    done = COALESCE($3, done)
WHERE id = $1
RETURNING id, title, done;
```

Example. PATCH /tasks/4 with { "done": true }: the model sends [4, null, true], so `title = COALESCE(NULL, title)` keeps "Add pagination" and `done = COALESCE(TRUE, done)` sets it. With { "done": false } it sends [4, null, false], and false is a real value, so the task is reopened.

Common mistake. Writing `fields.done || null` instead of `fields.done ?? null` in the model. `||` treats false as missing, so { "done": false } becomes NULL, `COALESCE` keeps the old true, and the client can never reopen a task. `??` replaces only undefined and null.

Pagination: LIMIT, OFFSET and ORDER BY

LIMIT n keeps at most n rows and OFFSET k skips the first k, so page p of size n is `LIMIT n OFFSET (p - 1) × n`; it only works with an ORDER BY that gives every row a fixed place.

- **Always ORDER BY:** without it the database may return rows in any order, and the order can change between two queries, so page 2 could repeat a row of page 1 and skip another.
- **Make the order unique:** sorting by `due_date`? Add `id` as a tie-breaker (`ORDER BY due_date, id`), because many tasks share a date.
- **Cap the limit** (say at 100), or `?limit=1000000` returns everything anyway (see [Pagination limits as protection](#)).
- **Deep pages get slow:** `OFFSET` still reads the rows it skips. Large sites switch to `keyset pagination` (`WHERE id > last_seen_id`), which you do not need here.

SQL

```
-- page 1, 2, 3 of 4 tasks each
SELECT id, title FROM tasks ORDER BY id LIMIT 4 OFFSET 0;  -- ids 1-4
SELECT id, title FROM tasks ORDER BY id LIMIT 4 OFFSET 4;  -- ids 5-8
SELECT id, title FROM tasks ORDER BY id LIMIT 4 OFFSET 8;  -- id 9

-- the model, with parameters (pg)
SELECT id, user_id, title, done FROM tasks
WHERE user_id = $1
```

```
ORDER BY id
LIMIT $2 OFFSET $3;
```

Example. GET /tasks?page=2&limit=4: the model computes $\text{offset} = (2 - 1) \times 4 = 4$, runs ... ORDER BY id LIMIT \$1 OFFSET \$2 with [4, 4] and returns tasks 5, 6, 7 and 8. Page 3 returns only task 9: a short page tells the client it reached the end.

Common mistake. Computing the offset from the page number as $\text{page} \times \text{size}$. Pages are usually numbered from 1, so page 1 must skip 0 rows: $\text{offset} = (\text{page} - 1) \times \text{size}$. With $\text{page} \times \text{size}$, the first page is never shown.

Joins and aggregates

JOIN: combining tables

A **JOIN** builds one result from two tables by pairing the rows that satisfy the **ON** condition, usually "foreign key = primary key"; an **inner join** (plain JOIN) keeps only the rows that found a partner.

Aliases tasks t, users u: short names that say which table a column comes from (t.title, u.email).

Ambiguous columns When both tables have a column called id, prefix it, or the database answers "column reference id is ambiguous".

Chained joins tasks → task_tags → tags follows a many-to-many relationship through its join table.

After the joins WHERE, ORDER BY and LIMIT work on the combined rows as on any table.

SQL

```
SELECT t.id, t.title, u.email
FROM tasks t
JOIN users u ON u.id = t.user_id
WHERE u.email = 'ana@example.com' AND t.done = FALSE
ORDER BY t.id DESC;
```

```
-- through the join table (many-to-many)
SELECT t.title, g.name AS tag
FROM tasks t
JOIN task_tags tt ON tt.task_id = t.id
JOIN tags g      ON g.id = tt.tag_id;
```

Example. The first query pairs 9 tasks with their owners (9 combined rows), keeps ana's 3 open ones and sorts them newest first: tasks 4, 3 and 1, each with ana@example.com. dan, who owns no tasks, appears in no combined row, so a JOIN can never list him: that needs a LEFT JOIN.

Common mistake. Forgetting the ON condition, or writing FROM tasks, users with no condition. Every task is then paired with **every** user ($9 \times 4 = 36$ rows of nonsense), a "cross join". If a join returns far more rows than either table has, check the ON.

LEFT JOIN: keeping rows without a match

A **LEFT JOIN** keeps **every** row of the table on its left; when a row has no partner on the right, the right-hand columns are filled with NULL.

Using the padded row

You want	Write	Why
Users with no tasks	WHERE t.id IS NULL	The task's primary key is NULL only in a padded row
Tasks per user, zeros included	COUNT(t.id)	It skips the NULL, so dan gets 0; COUNT(*) would count his row and give 1
A filter that keeps unmatched users	In the ON: ON t.user_id = u.id AND t.done = FALSE	In the WHERE, NULL is not FALSE: the padded rows go and the join turns inner

SQL

```
-- every user and how many tasks they own, 0 included
SELECT u.email, COUNT(t.id) AS task_count
FROM users u
LEFT JOIN tasks t ON t.user_id = u.id
GROUP BY u.id, u.email;

-- users with no tasks at all
SELECT u.email
FROM users u
LEFT JOIN tasks t ON t.user_id = u.id
WHERE t.id IS NULL;

-- open tasks per user, still keeping users with none
SELECT u.email, COUNT(t.id) AS open_tasks
FROM users u
LEFT JOIN tasks t ON t.user_id = u.id AND t.done = FALSE
GROUP BY u.id, u.email;
```

Example. On the sample data the first query gives ana 4, ben 3, cleo 2 and dan 0; the second returns only dan@example.com. This "count per parent, zeros included" report is the classic reason to reach for `LEFT JOIN`. In the SQL runner, try the challenges "Users with no tasks" and "Tasks per user, zeros included".

Common mistake. Using `COUNT(*)` with a `LEFT JOIN` to count children. The padded row is still a row, so a user with no tasks is reported with 1 task. Count a column of the right-hand table, such as `COUNT(t.id)`.

COUNT and GROUP BY

Aggregate functions (`COUNT`, `SUM`, `AVG`, `MIN`, `MAX`) turn many rows into one value; `GROUP BY` makes one group per distinct value and computes the aggregates once per group; `HAVING` filters the groups.



Rows are filtered, grouped, and then the groups are filtered.

Two filters, before and after grouping

	WHERE	HAVING
Filters	Rows, before grouping	Groups, after counting
Example	Only the open tasks	Only users with 2 or more
Can use COUNT(*)	No: the groups do not exist yet	Yes

SQL

```
-- open tasks per user, busiest first, only users with 2 or more
SELECT u.email, COUNT(*) AS open_tasks
FROM tasks t
JOIN users u ON u.id = t.user_id
WHERE t.done = FALSE          -- rows: keep the open tasks
GROUP BY u.id, u.email        -- one group per user
HAVING COUNT(*) >= 2          -- groups: keep the busy users
ORDER BY open_tasks DESC;
```

Example. On the sample data the groups are ana (3 open tasks), ben (2) and cleo (1); `HAVING COUNT(*) >= 2` drops cleo, so the result is ana@example.com 3, ben@example.com 2. "Who has the most open tasks?" is the same query with `LIMIT 1` instead of `HAVING` (with a tie, `LIMIT 1` silently hides the other users).

Common mistake. Expecting a number from pg to arrive as a JavaScript number. `COUNT(*)` returns a bigint, and pg sends bigint to JavaScript as a **string** ("3") so huge values are not rounded. Convert it in the model (`Number(row.open_tasks)`) or in SQL (`COUNT(*)::int`) before it reaches the JSON.

Indexes and migrations

Indexes: finding rows fast

An **index** is an extra sorted structure over one or more columns that lets the database jump to the matching rows instead of reading the whole table; it speeds up reads on those columns and slows writes a little.

Which columns have an index

Column	Index
Primary key	Created automatically
UNIQUE column	Created automatically: that is how duplicates are found fast
Foreign key	PostgreSQL: not automatic, add one (tasks.user_id). MySQL: automatic
A column you filter, join or sort by	Add one when a query on it is slow and frequent

- **Not free:** each index takes disk space, and every INSERT, UPDATE and DELETE must update every index of the table. No index "on every column just in case".
- **Check it:** EXPLAIN (PostgreSQL) or EXPLAIN QUERY PLAN (SQLite) shows whether a query uses an index.

SQL (PostgreSQL)

```
CREATE INDEX tasks_user_id_idx ON tasks (user_id);

-- PostgreSQL: how will this query run? (on a big table)
EXPLAIN SELECT id, title FROM tasks WHERE user_id = 2;
-- Index Scan using tasks_user_id_idx on tasks ...      (with the index)
-- Seq Scan on tasks  Filter: (user_id = 2)              (without it)
```

Example. The sample database has tasks_user_id_idx. In the SQL runner, the "Is an index used?" example shows SEARCH tasks USING INDEX tasks_user_id_idx (user_id=?) for WHERE user_id = 2, but SCAN tasks for WHERE title = 'Set up CI', because no index covers title. A GET /tasks?userId=2 on a big table is fast only in the first case.

Common mistake. Believing an index changes what a query returns, or that it makes everything faster. An index only changes **how fast** the same rows are found, and only for queries that filter or sort by the indexed columns; on a table that is mostly written to, extra indexes make it slower.

Migrations: versioning the schema

A **migration** is a numbered script that changes the schema one step (create a table, add a column); the scripts live with the code and a tool runs, in order, the ones a database has not run yet.

1. The migration tool keeps a table in the database that lists the migrations already run there.
2. Running "migrate" compares that list with the files.
3. It runs only the missing ones, in number order, and records each one.

SQL (PostgreSQL)

```
-- migrations/001_create_users.sql
CREATE TABLE users (
  id    SERIAL PRIMARY KEY,
  email TEXT NOT NULL UNIQUE
);

-- migrations/002_create_tasks.sql
CREATE TABLE tasks (
  id        SERIAL PRIMARY KEY,
  user_id INT NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  title    TEXT NOT NULL,
  done     BOOLEAN NOT NULL DEFAULT FALSE
);

-- migrations/003_add_due_date_to_tasks.sql
ALTER TABLE tasks ADD COLUMN due_date DATE;
```

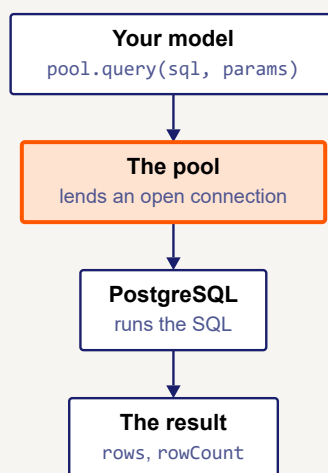
Example. Your production database has run 001 and 002. You push a feature with migration 003. The deploy runs "migrate": the tool sees 001 and 002 in its history table, runs only 003 (ALTER TABLE ... ADD COLUMN due_date), records it, and existing tasks get due_date = NULL. A setup script that starts with DROP TABLE IF EXISTS and recreates every table is fine for a throwaway practice database, but it wipes the data each time it runs, which is exactly what migrations avoid.

Common mistake. Fixing a typo in 002_create_tasks.sql after it ran on your teammate's database. Their migration history already lists 002, so the fix is never applied there and the two databases silently differ. Add 004_fix_....sql instead.

From Node to SQL

Connecting from Node: drivers and pools

A **driver** is the npm package that speaks a database's network protocol (pg for PostgreSQL, mysql2 for MySQL); a **pool** keeps a few open connections and lends one to each query, so requests do not pay for a new connection every time.



One pool, created at start-up, lends an open connection to every query.

DATABASE_URL	The connection details (host, port, user, password, database) from the environment, never from the code: postgres://app:secret@localhost:5432/tasks in .env (see Environment variables).
pool.query(text, params)	Runs one statement and returns a promise of the result.
result.rows	An array of plain objects, one per row, with the column names as keys.
result.rowCount	How many rows were returned or changed.
\$1, \$2...	Placeholders: the values in params travel apart from the SQL text. mysql2 writes ?.

JavaScript (Node)

```

// src/db/pool.js - created once, shared by every model
const { Pool } = require("pg");
const pool = new Pool({ connectionString: process.env.DATABASE_URL });
module.exports = pool;

// anywhere in a model
const result = await pool.query("SELECT id, title FROM tasks WHERE id = $1", [7]);
result.rows; // [ { id: 7, title: "Draft project plan" } ]
result.rowCount; // 1

// MySQL with mysql2: "?" placeholders, the result is [rows, fields]
const mysql = require("mysql2/promise");

```

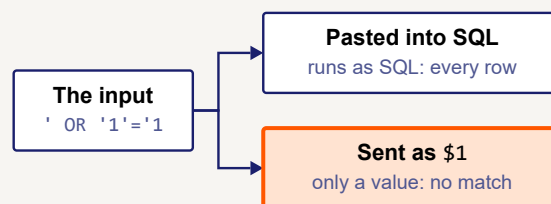
```
const db = mysql.createPool({ uri: process.env.DATABASE_URL });
const [rows] = await db.query("SELECT id, title FROM tasks WHERE id = ?", [7]);
```

Example. A burst of 50 requests arrives. With a pool of 10, the first 10 queries get a connection at once and the others wait a few milliseconds for one to be handed back; the database sees at most 10 connections. Opening a fresh connection per request would mean 50 logins and could exceed the database's connection limit.

Common mistake. Writing `new Pool()` inside a request handler or a model function. Each call opens new connections that are never closed, and after a few hundred requests the database refuses with "too many clients". Create the pool once in its own module and `require` it, or hand it to the model (see [The models layer](#)).

SQL injection and parameterised queries

SQL injection happens when user input is pasted into the text of an SQL statement, so the input can change what the statement does; **parameterised queries** send the SQL text and the values separately (\$1 in pg, ? in mysql2), so input is always just a value.



The same input: part of the query, or just a value.

- **Worse inputs:** with pg, a query without parameters may contain several statements, so `x'; DELETE FROM tasks; --` deletes the table (-- comments out the leftover quote).
- **Honest users break it too:** the name O'Brien ends the string early.

JavaScript (Node, pg)

```
// VULNERABLE: the input becomes SQL
const sql = `SELECT id, title FROM tasks WHERE user_id = ${req.query.user}`;
await pool.query(sql);           // ?user=1 OR 1=1 → every task

// SAFE: fixed SQL text + values apart
await pool.query("SELECT id, title FROM tasks WHERE user_id = $1", [req.query.user]);
// mysql2: db.query("... WHERE user_id = ?", [req.query.user])

// Identifiers cannot be parameters: use an allow-list
const SORTABLE = { id: "id", title: "title", due: "due_date" };
const column = SORTABLE[req.query.sort] || "id";    // never the raw input
await pool.query(`SELECT id, title FROM tasks ORDER BY ${column} LIMIT $1`, [limit]);
```

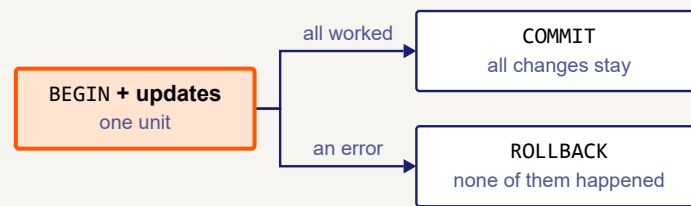
Example. In the SQL runner's injection demo, type `' OR '1'='1'`: the concatenated query returns all 9 tasks; the `$1` query returns none, because no user has that strange email. Then try `x'; DELETE FROM tasks; --` and look at "tasks left afterwards": 0 for concatenation, 9 for the placeholder.

Common mistake. Thinking a template literal (a backtick string with `${id}` inside) is safer than `+`. It is still string concatenation: the input still becomes part of the SQL text. Only a placeholder plus a params array keeps it out. Escaping quotes by hand is not enough either: it is easy to miss a case, and numbers need no quotes to inject (`1 OR 1=1`).

Transactions: all or nothing

A **transaction** groups several statements so that they all take effect (`COMMIT`) or none does (`ROLLBACK`); other

requests never see the half-finished state.



All or nothing: a transaction ends in COMMIT or ROLLBACK.

In SQL

```

BEGIN;
UPDATE accounts SET balance = balance - 50 WHERE id = 1;
UPDATE accounts SET balance = balance + 50 WHERE id = 2;
COMMIT;      -- or ROLLBACK; to undo both
  
```

JavaScript (Node, pg)

```

const client = await pool.connect();          // one connection for the whole unit
try {
  await client.query("BEGIN");
  await client.query("UPDATE accounts SET balance = balance - $1 WHERE id = $2", [50, fromId]);
  await client.query("UPDATE accounts SET balance = balance + $1 WHERE id = $2", [50, toId]);
  await client.query("COMMIT");                // both, or...
} catch (err) {
  await client.query("ROLLBACK");              // ...neither
  throw err;
} finally {
  client.release();                            // back to the pool, always
}
  
```

Example. In the SQL runner, the "Transaction" example runs BEGIN, marks all of ana's tasks done, shows them done, then ROLLBACKS: the last SELECT shows them open again, as if the update never happened. Replace ROLLBACK with COMMIT and the change stays.

Common mistake. Writing `await pool.query("BEGIN")` and then more `pool.query` calls. Each call may borrow a different connection from the pool, so the statements are not inside your transaction, and ROLLBACK undoes nothing. Use `pool.connect()` and one `client` from BEGIN to COMMIT.

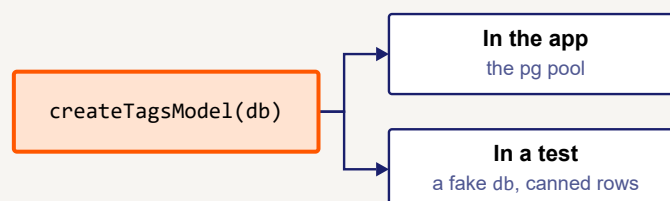
The models layer: where the SQL lives

A model is a module of async functions, one per data operation (`findAll`, `findById`, `create`, `update`, `remove...`); each wraps one parameterised query and returns plain data: an object, an array, null or true/false.

Who does what in the Tasks API

Layer	File (typical layout)	Its job	Never
Routes	<code>src/routes/tasks.js</code>	<code>router.get("/:id", controller.getTask)</code>	Contain logic
Controller	<code>src/controllers/tasksController.js</code>	Read req, validate, call the model, choose status + JSON	Write SQL
Model	<code>src/models/tasksModel.js</code>	Run parameterised SQL, return plain objects or null	Touch req / res

A factory that is handed its database



One model, any database object: the real pool in the app, a fake in tests.

1. Write the SQL with placeholders.
 2. `await db.query(sql, params).`
 3. Shape the answer: rows for a list, `rows[0] ?? null` for one item, `rowCount > 0` for "did it delete?".
- **Errors pass through:** the model does not catch them; the controller does, and in Express 4 calls `next(err)` (see [Async handlers: Express 4 versus Express 5](#)).
 - **No SQL in routes:** `pool.query` straight from route handlers means the same query written five times, a column rename hunted through the whole app, and routes that cannot be tested without a database.

JavaScript (Node)

```
// src/models/tagsModel.js – the same pattern as a tasksModel
function createTagsModel(db) {
  return {
    async findAll() {
      const { rows } = await db.query("SELECT id, name FROM tags ORDER BY name", []);
      return rows;
    },
    async findByName(name) {
      const { rows } = await db.query("SELECT id, name FROM tags WHERE name = $1", [name]);
      return rows[0] ?? null; // one object, or null
    },
    async create(name) {
      const { rows } = await db.query(
        "INSERT INTO tags (name) VALUES ($1) RETURNING id, name", [name]);
      return rows[0];
    },
  };
}
module.exports = { createTagsModel };

// controller (Express 4): await the model, forward failures
async function getTag(req, res, next) {
  try {
    const tag = await tagsModel.findByName(req.params.name);
    if (!tag) return res.status(404).json({ error: "Tag not found" });
    res.json(tag);
  } catch (err) {
    next(err);
  }
}
```

Example. A test needs no database: `const fake = { query: async () => ({ rows: [{ id: 1, name: "urgent" }], rowCount: 1 })`; `const tags = createTagsModel(fake);` and `await tags.findByName("urgent")` returns `{ id: 1, name: "urgent" }`. In the app the same model gets the pool: `createTagsModel(require("./db/pool"))`.

Common mistake. Letting the model decide HTTP things: `if (!row) res.status(404)...` inside `findById`, or throwing an error for "not found". A missing row is a normal answer, `null`; the controller turns it into 404. Keeping HTTP out of the model is what lets the same model serve a route, a script and a test.

From rows to JSON

A database row and the JSON an API returns are two shapes: one mapping function turns `snake_case` columns into `camelCase` keys, converts the types `pg` returns and lists only the fields the API sends.

What `pg` returns and what the JSON needs

In the row	pg gives	Map it to
Column names	<code>snake_case</code> (<code>user_id</code> , <code>created_at</code>): many databases fold unquoted names to lower case	<code>camelCase</code> keys (<code>userId</code> , <code>createdAt</code>)
BIGINT, NUMERIC (such as <code>COUNT(*)</code>)	A string ("4"), so huge values are not rounded	<code>Number(row.task_count)</code> , or <code>COUNT(*)::int</code> in SQL

In the row	pg gives	Map it to
TIMESTAMP	A Date object	An ISO string: row.created_at.toISOString()
Secret columns (password_hash)	Whatever SELECT asked for	Nothing: list only the fields to send

```
// A row as pg returns it from SELECT * FROM users
const row = {
  id: 1,
  email: "ana@example.com",
  password_hash: "$2b$10$Qe3...",
  created_at: new Date("2026-09-01T09:00:00Z"),
  task_count: "4", // COUNT(*) is a bigint: pg sends a string
};

// Opt-in mapping: only the listed fields leave the server
function toUserDto(r) {
  return {
    id: r.id,
    email: r.email,
    createdAt: r.created_at.toISOString(),
    taskCount: Number(r.task_count),
  };
}

console.log(toUserDto(row));
console.log("password_hash" in toUserDto(row)); // false
```

Example. Run the Try it box: the DTO has createdAt, a real number taskCount: 4, and no password_hash. Without it, res.json(row) would send password_hash, created_at and "task_count": "4" (a string) to every client, and the front end would break the day someone renames a column.

Common mistake. Removing secrets with delete row.password_hash before res.json(row). It works until someone adds another sensitive column, or forgets the delete in one route. An explicit list of the fields to send ({ id, email, createdAt }) cannot leak what it does not name.